

UNIVERSIDAD DE GUADALAJARA

Centro Universitario de Ciencias Exactas e Ingenierías

REPORTE DE PRÁCTICA

Proyecto Final — Fase 1

Diseño de Datapath MIPS 32 bits — Instrucciones Tipo R

Materia:	Arquitectura de Computadoras
Profesor:	Jorge Ernesto López Arce Delgado
Integrantes:	Diego Israel González Sánchez Derek Gabriel Casillas López Iván Samuel Ortiz Orellana Omar Luna Reyes
Fecha:	09 de mayo de 2026

1. INTRODUCCIÓN

1.1 Arquitectura MIPS y paradigma RISC

El procesador MIPS (Microprocessor without Interlocked Pipeline Stages) fue desarrollado en la Universidad de Stanford por John Hennessy a principios de los años 80. Forma parte de la familia de arquitecturas RISC (Reduced Instruction Set Computer), cuya filosofía consiste en diseñar un conjunto pequeño de instrucciones simples y ortogonales que puedan ejecutarse en un único ciclo de reloj con rutas de datos eficientes. Esta simplicidad facilita implementaciones en hardware y convierte a MIPS en una plataforma de referencia para la enseñanza de arquitectura de computadoras.

La versión de 32 bits del procesador MIPS maneja palabras de 32 bits y cuenta con 32 registros de propósito general (R0–R31), donde R0 está cableado permanentemente a cero. Su diseño separa la memoria de instrucciones de la memoria de datos (arquitectura Harvard modificada) y emplea un contador de programa (PC) que se incrementa en 4 bytes por instrucción, ya que todas las instrucciones tienen longitud fija de 32 bits.

1.2 Componentes principales del datapath

El datapath de ciclo único (single-cycle) que se implementa en este proyecto integra los siguientes bloques funcionales:

- **Contador de Programa (PC):** Registro que almacena la dirección de la instrucción actual. Se incrementa en 4 en cada ciclo.
- **Memoria de Instrucciones (IM):** Memoria de solo lectura que contiene el programa en binario, indexada por el valor del PC.
- **Banco de Registros (RF):** Arreglo de 32 registros de 32 bits con dos puertos de lectura simultánea y un puerto de escritura.
- **Unidad Aritmético-Lógica (ALU):** Realiza las operaciones indicadas por la señal `alu_control`, incluyendo suma, resta, AND, OR y comparaciones.
- **Unidad de Control:** Decodifica el campo opcode de la instrucción y genera señales de control para el resto del datapath.
- **Control de la ALU:** Decodifica el campo `funct` de instrucciones tipo R y combina con la señal `ALUOp` para determinar la operación exacta.
- **Multiplexores (MUX):** Seleccionan rutas de datos alternativas según las señales de control (registro destino `rd` vs `rt`, y resultado ALU hacia el banco de registros).

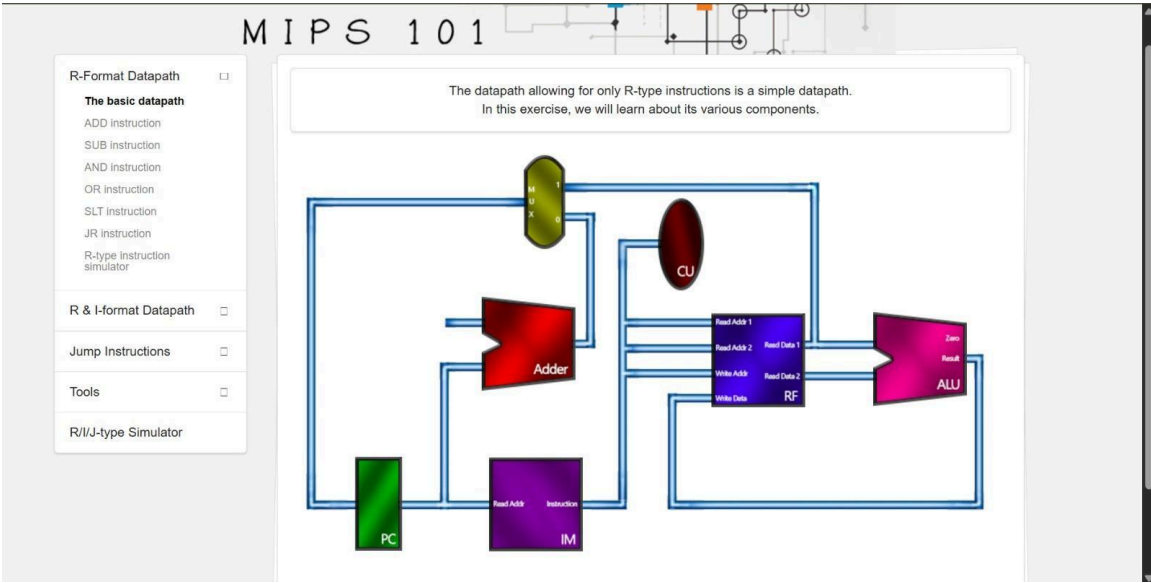


Figura 1. Diagrama del datapath MIPS para instrucciones tipo R, mostrando PC, IM, Adder, RF, CU y ALU interconectados.

Las instrucciones MIPS de 32 bits se clasifican en tres formatos según la función que realizan:

Tipo	Descripción	Formato (bit [31..0])
R	Operaciones entre registros	opcode(6) rs(5) rt(5) rd(5) shamt(5) funct(6)

1.4 Formato detallado de instrucción tipo R

Las instrucciones tipo R son el foco de esta Fase 1. Tienen opcode siempre igual a **000000**, y la operación específica la determina el campo **funct** de 6 bits. El campo **rd** es el registro destino, mientras que **rs** y **rt** son los registros fuente. El campo **shamt** (shift amount) se utiliza únicamente en instrucciones de desplazamiento.

Campo	opcode	rs	rt	rd	shamt	funct
Bits	31–26	25–21	20–16	15–11	10–6	5–0
Ancho	6 bits	5 bits	5 bits	5 bits	5 bits	6 bits
Valor	000000	fuelle1	fuelle2	destino	despl.	operación

Sintaxis ensamblador tipo R: **instrucción \$rd, \$rs, \$rt** (excepción: desplazamientos usan \$rd, \$rt, shamt).

1.5 Set de instrucciones implementado

La tabla siguiente muestra las instrucciones contempladas en el diseño, con su tipo, código de función (funct) cuando aplica, y la sintaxis en ensamblador MIPS:

Instrucción	Tipo	Opcode	Funct	Sintaxis
add	R	000000	100000	add \$rd, \$rs, \$rt
sub	R	000000	100010	sub \$rd, \$rs, \$rt
and	R	000000	100100	and \$rd, \$rs, \$rt

or	R	000000	100101	or \$rd, \$rs, \$rt
nor	R	000000	100111	nor \$rd, \$rs, \$rt
xor	R	000000	100110	xor \$rd, \$rs, \$rt
slt	R	000000	101010	slt \$rd, \$rs, \$rt
sll	R	000000	000000	sll \$rd, \$rt, shamt
srl	R	000000	000010	srl \$rd, \$rt, shamt
nop	R	000000	000000	nop (sll \$0, \$0, 0)

1.6 Análisis del ISA: algoritmos posibles en ensamblador

Con base en el conjunto de instrucciones disponibles, es posible analizar qué categorías de algoritmos pueden implementarse y cuáles están fuera del alcance:

Algoritmos factibles con este ISA:

- **Máximo Común Divisor (Algoritmo de Euclides por restas):** Usa sub, slt y j. Compara dos valores y resta el menor del mayor hasta que ambos sean iguales. Compatible al 100% con las instrucciones tipo R implementadas.
- **Comparación de máximo y mínimo:** Usando slt es posible determinar cuál de dos valores es mayor o menor, e implementar una búsqueda del máximo en un conjunto de registros.
- **Operaciones lógicas combinadas:** Con and, or, xor y nor se pueden implementar funciones booleanas, máscaras de bits y comprobaciones de flags sobre valores almacenados en registros.
- **Operaciones de desplazamiento:** sll y srl permiten multiplicar o dividir por potencias de 2 de forma eficiente, útil para empaquetado/desempaquetado de bits.
- **Suma y resta de registros:** add y sub permiten cálculos aritméticos simples y la simulación de operaciones más complejas mediante combinaciones de sumas y restas sucesivas.
- **Multiplicación por sumas repetidas (simulada):** Sin instrucción mul, se puede multiplicar $A \times B$ sumando A repetidamente B veces usando únicamente add y slt con j para el ciclo.

Limitaciones del ISA en Fase 1:

- No se incluyen instrucciones de acceso a memoria ni con inmediato, por lo que todos los operandos deben estar precargados en registros.
- La ausencia de mul y div directas obliga a reemplazarlas con algoritmos de software más lentos (sumas o restas repetidas).
- El rango de valores está limitado a 32 bits con signo (-2,147,483,648 a 2,147,483,647).
- Sin instrucciones de salto condicional en el hardware de Fase 1, los bucles dinámicos no pueden simularse directamente sobre el datapath actual.

2. OBJETIVO

2.1 Objetivo General

Diseñar e implementar un datapath de ciclo único para un procesador MIPS de 32 bits utilizando el lenguaje de descripción de hardware Verilog, capaz de ejecutar correctamente instrucciones tipo R, validando su funcionamiento mediante simulación en ModelSim y proponiendo un algoritmo en lenguaje ensamblador que aproveche el conjunto de instrucciones definido.

2.2 Objetivos Particulares

- Analizar la arquitectura MIPS de 32 bits, su filosofía RISC y sus componentes internos.
- Comprender el formato binario de las instrucciones tipo R y la función de cada campo (opcode, rs, rt, rd, shamt, funct).
- Diseñar e implementar en Verilog los módulos individuales del datapath: PC, sumador, memoria de instrucciones, banco de registros, ALU, control de ALU, unidad de control, multiplexores, extensor de signo y memoria de datos.
- Interconectar los módulos en el módulo superior MIPS_Top para formar el datapath completo de ciclo único.
- Precargar la memoria de instrucciones y el banco de registros con valores de prueba y verificar la ejecución correcta de instrucciones tipo R mediante simulación.
- Proponer un algoritmo en lenguaje ensamblador MIPS que sea compatible con el ISA implementado, justificando su elección con base en las instrucciones disponibles.

3. DESARROLLO

3.1 Arquitectura del datapath single-cycle

El diseño implementado corresponde a un datapath de ciclo único, donde cada instrucción completa su ejecución en un único ciclo de reloj. La ruta de datos sigue el flujo clásico: Fetch → Decode → Execute → Memory → Write-Back, todo en el mismo ciclo. Se implementaron 10 módulos independientes en Verilog, cada uno correspondiente a un bloque funcional claramente definido, más un módulo superior de interconexión (MIPS_Top) y un testbench.

3.2 Módulo PC — Contador de Programa

El módulo PC es un registro síncrono de 32 bits que almacena la dirección de la instrucción actual. En cada flanco positivo de reloj, carga el valor de next_pc, que proviene del sumador PC+4. La señal de reset lo lleva a la dirección 0 para iniciar la ejecución.

```
// Program Counter
module PC(
    input clk,
    input reset,
    input [31:0] next_pc,
    output reg [31:0] pc
);
    always @(posedge clk or posedge reset) begin
        if (reset)
            pc <= 0;
        else
            pc <= next_pc;
        end
    endmodule
```

3.3 Módulo Adder — Sumador PC+4

Sumador combinacional de 32 bits que calcula la dirección de la siguiente instrucción sumando 4 al valor actual del PC. Dado que todas las instrucciones MIPS tienen 32 bits (4 bytes), el incremento fijo de 4 garantiza que el PC apunte a la siguiente instrucción en memoria.

```
// Sumador para incrementar el PC
module Adder(
    input [31:0] a,
    input [31:0] b,
    output [31:0] result
);
    assign result = a + b;
endmodule
```

3.4 Módulo InstructionMemory — Memoria de Instrucciones

Memoria de solo lectura de 256 palabras de 32 bits. El arreglo se inicializa con las instrucciones de prueba cargadas desde el archivo *TestF1_MemInst.mem* mediante la directiva *\$readmemb*. La dirección se obtiene de los bits [9:2] del PC, descartando los 2 bits menos significativos para garantizar acceso alineado a palabra de 4 bytes.

```
// Memoria de Instrucciones (ROM)
module InstructionMemory(
    input [31:0] address,
    output [31:0] instruction
);
    reg [31:0] mem [0:255];

    initial begin
        $readmemb("TestF1_MemInst.mem", mem);
    end

    // Lectura alineada a palabra (descarta 2 bits LSB)
    assign instruction = mem[address[9:2]];
endmodule
```

3.5 Módulo RegisterFile — Banco de Registros

El banco de registros contiene 32 registros de 32 bits cada uno. Soporta la lectura combinacional simultánea de dos registros (rs y rt) y la escritura síncrona en un registro destino (rd). Se protege el registro \$0 de escritura para

mantenerlo siempre en cero, conforme a la especificación MIPS. El contenido inicial de los registros se carga desde el archivo *TestF1_BReg.mem*, que inicializa el registro N con el valor N (R0=0, R1=1, ..., R31=31).

```
// Banco de Registros (32 registros de 32 bits)
module RegisterFile{
    input clk, RegWrite, input
    [4:0] rs, rt, rd, input
    [31:0] write_data,
    output [31:0] read_data1, read_data2
};

reg [31:0] registers [0:31];

initial begin
    $readmemb("TestF1_BReg.mem", registers);
end

// Lectura combinacional (asíncrona)
assign read_data1 = registers[rs]; assign
read_data2 = registers[rt];

// Escritura síncrona (protegiendo $zero)
always @(posedge clk) begin
    if (RegWrite && rd != 5'd0) registers[rd]
        <= write_data;
end

endmodule
```

3.6 Módulo ALU — Unidad Aritmético-Lógica

La ALU realiza la operación indicada por la señal de 4 bits *alu_control*. Además de las operaciones aritméticas básicas (add, sub) y lógicas (and, or, xor, nor), incluye la comparación *slt* (set if less than) que devuelve 1 si $a < b$, y las instrucciones de desplazamiento *sll* y *srl* que utilizan el campo *shamt*. La señal *zero* se activa cuando el resultado es cero, lo que se usa para implementar instrucciones de salto condicional como beq.

```
// Unidad Aritmético-Lógica
module ALU(
    input [31:0] a, b, input
    [4:0] shamt,
    input [3:0] alu_control, output
    reg [31:0] result, output zero
);

always @(*) begin
    case (alu_control)
        4'b0000: result = a & b;
        4'b0001: result = a | b; // AND
        4'b0010: result = a + b; // OR
        4'b0110: result = a - b; // ADD
        // SUB
        4'b0111: result = (a < b) ? 32'd1 : 32'd0; // SLT
        4'b1100: result = ~(a | b); // NOR
        4'b0011: result = a ^ b; // XOR
        4'b1000: result = b << shamt; // SLL
        4'b1001: result = b >> shamt; // SRL default:
        result = 32'd0;
    endcase
    end
    assign zero = (result == 0);
endmodule
```

3.7 Módulo ALUControl — Control de la ALU

Este módulo decodifica el campo *funct* de 6 bits de las instrucciones tipo R para generar la señal *alu_control* de 4 bits que indica a la ALU qué operación ejecutar. Cuando *ALUOp* = 2'b10 (instrucciones tipo R), el módulo interpreta el campo *funct* y mapea cada código a la señal de control correspondiente.

```
// Control de la ALU
module ALUControl{
    input [5:0] funct,
    input [1:0] ALUOp,
    output reg [3:0] alu_control
};

always @(*) begin
    if (ALUOp == 2'b10) begin // Instrucciones Tipo R
        case (funct)
            6'b100000: alu_control = 4'b0010; // add
            6'b100010: alu_control = 4'b0110; // sub
            6'b100100: alu_control = 4'b0000; // and
            6'b100101: alu_control = 4'b0001; // or
            6'b101010: alu_control = 4'b0111; // slt
            6'b100111: alu_control = 4'b1100; // nor
            6'b100110: alu_control = 4'b0011; // xor
            6'b000000: alu_control = 4'b1000; // sll
            6'b000010: alu_control = 4'b1001; // srl
            default: alu_control = 4'b0000;
        endcase
    end else begin
        alu_control = 4'b0000;
    end
end
endmodule
```

3.8 Módulo Control — Unidad de Control Principal

La Unidad de Control decodifica el campo *opcode* de 6 bits de la instrucción. El único opcode manejado es **000000** (instrucciones tipo R), el cual activa *RegDst=1* (el destino es el campo rd), *RegWrite=1* (escritura habilitada) y *ALUOp=2'b10* (modo tipo R). Todas las demás señales se inicializan en 0 por defecto para evitar la generación de latches no deseados.

```
// Unidad de Control Principal
module Control(
    input [5:0] opcode,
    output reg RegDst, ALUSrc, MemtoReg,
    output reg RegWrite, MemRead, MemWrite,
    output reg [1:0] ALUOp
);
    always @(*) begin
        // Valores por defecto (evitan latches)
        {RegDst,ALUSrc,MemtoReg,RegWrite,MemRead,MemWrite} = 6'b0; ALUOp
        = 2'b00;

        case (opcode)
            6'b000000: begin // Tipo R RegDst
                = 1;
                RegWrite = 1;
                ALUOp = 2'b10;
            end
            default: RegWrite = 0;
        endcase
    end
endmodule
```

3.9 Módulo Mux2 — Multiplexor parametrizable

El multiplexor parametrizable *Mux2* selecciona entre dos entradas según la señal *sel*. Se instancia dos veces en el datapath: con ancho de 5 bits para seleccionar el registro destino (*rd* cuando *RegDst*=1, *rt* cuando *RegDst*=0), y con ancho de 32 bits para seleccionar el dato a escribir en el banco de registros (resultado de la ALU).

```
// Multiplexor 2:1 parametrizable module
Mux2 #(parameter WIDTH = 32) (
    input [WIDTH-1:0] d0, d1, input
    sel,
    output [WIDTH-1:0] out
);
    assign out = sel ? d1 : d0;
endmodule
```

3.10 Módulo MIPS_Top — Interconexión del datapath

MIPS_Top es el módulo superior que instancia y conecta todos los componentes mediante cables (*wire*). Las señales de control generadas por la Unidad de Control dirigen los multiplexores y habilitan lecturas/escrituras en registros y memoria. El flujo de ejecución de una instrucción tipo R en este datapath es:

- El PC envía su dirección a InstructionMemory, que entrega la instrucción de 32 bits.
- El Adder calcula PC+4 y lo realimenta al PC para el siguiente ciclo.
- La Unidad de Control decodifica el opcode (000000 → tipo R) y activa *RegDst*=1, *RegWrite*=1, *ALUOp*=2'b10.
- El Mux *RegDst* selecciona *instruction*[15:11] (campo *rd*) como dirección de escritura.
- El RegisterFile lee los registros *rs* (*instruction*[25:21]) y *rt* (*instruction*[20:16]).
- ALUControl decodifica el campo *funct* e indica a la ALU la operación a realizar.
- La ALU ejecuta la operación con los datos de *rs* y *rt*; el resultado queda en *alu_result*.
- El Mux *MemtoReg* (*sel*=0) selecciona *alu_result* como dato a escribir en el banco.
- En el siguiente flanco positivo de reloj, el resultado se escribe en el registro *rd*.

3.11 Archivos de prueba (.mem)

Para verificar el funcionamiento del procesador se utilizaron dos archivos de memoria precargada. El archivo *TestF1_BReg.mem* inicializa el banco de registros con valores del 0 al 31 en binario (registro N = valor N), lo que permite verificar fácilmente qué registros se leen y cuáles son modificados. El archivo *TestF1_MemInst.mem* contiene las instrucciones de prueba codificadas en binario, incluyendo operaciones add, sub, and, or y slt sobre los registros precargados.

3.12 Testbench y resultados de simulación

El testbench *tb_MIPS.v* instancia el módulo MIPS_Top, genera un reloj de 10 ns de periodo (5 ns por semiciclo) y aplica un pulso de reset inicial de 10 ns para llevar el PC a cero. Tras liberar el reset, el procesador comienza a ejecutar las instrucciones precargadas. El monitor del testbench imprime en cada ciclo el PC, la instrucción obtenida, el resultado de la ALU y el estado de la señal RegWrite.

```
// Testbench Global
`timescale 1ns/1ps
module tb_MIPS;
    reg clk, reset;
    MIPS_Top dut(.clk{clk}, .reset{reset});
    always #5 clk = ~clk;    // Reloj de 100 MHz (T=10ns)
    initial begin
        clk = 0; reset = 1;
        #10;
        reset = 0;
        #200; // 20 ciclos de ejecución
        $stop;
    end

    initial begin
        $monitor("t=%0t | PC=%h | Instr=%h | ALU=%h | WrReg=%b",
            $time, dut.pc, dut.instruction,
            dut.alu_result, dut.RegWrite);
    end
end
endmodule
```

En la simulación en ModelSim se observó que, durante el pulso de reset (0–10 ns), el PC permanece en 0x00000000 y no se ejecutan instrucciones. A partir del ciclo 1 (t=10 ns), el PC se incrementa de 4 en 4 y la señal RegWrite se activa en cada instrucción tipo R, confirmando que el resultado de la ALU se escribe correctamente en el registro destino. Las formas de onda capturadas en ModelSim muestran la correcta secuencia de valores en las señales PC, instruction, alu_result, write_reg y RegWrite, validando el funcionamiento del datapath single-cycle para instrucciones tipo R.

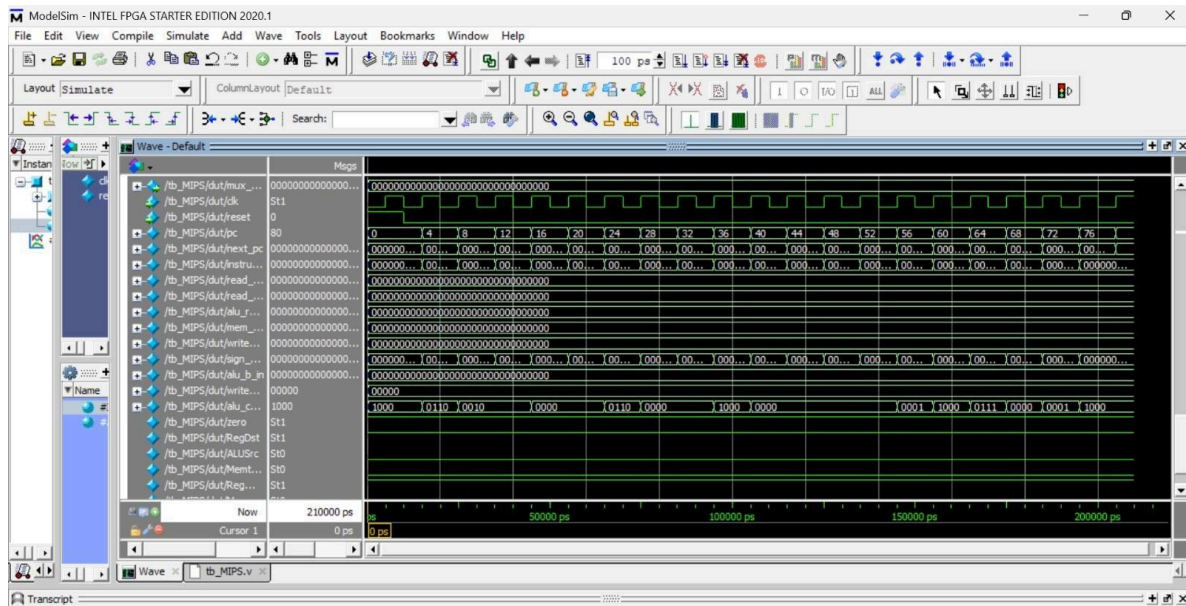


Figura 2. Simulación en ModelSim — señales del datapath: PC incrementando de 4 en 4 (0x00 → 0x04 → 0x08 → ...), señales de control RegDst=1, RegWrite=1, ALUSrc=0 y alu_control activo.

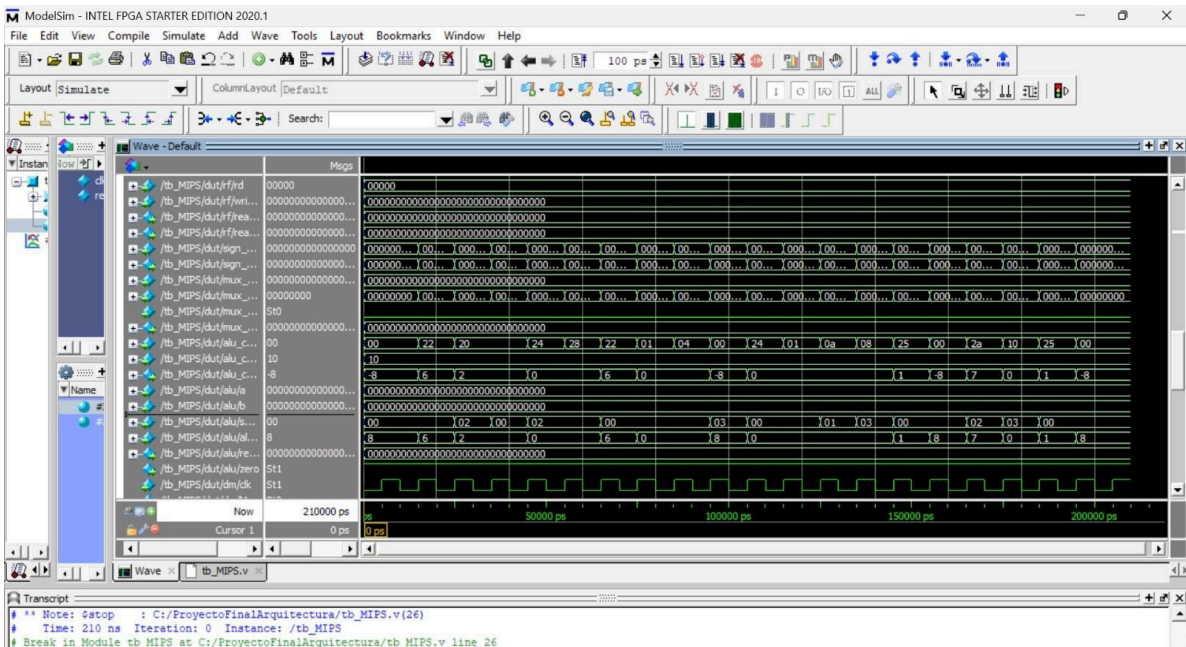


Figura 3. Simulación en ModelSim — detalle de la ALU: señales *alu_c* (código de operación), *alu_result* y *zero*. Se observan los resultados de instrucciones *add*, *sub*, *and*, *or* y *slt*.

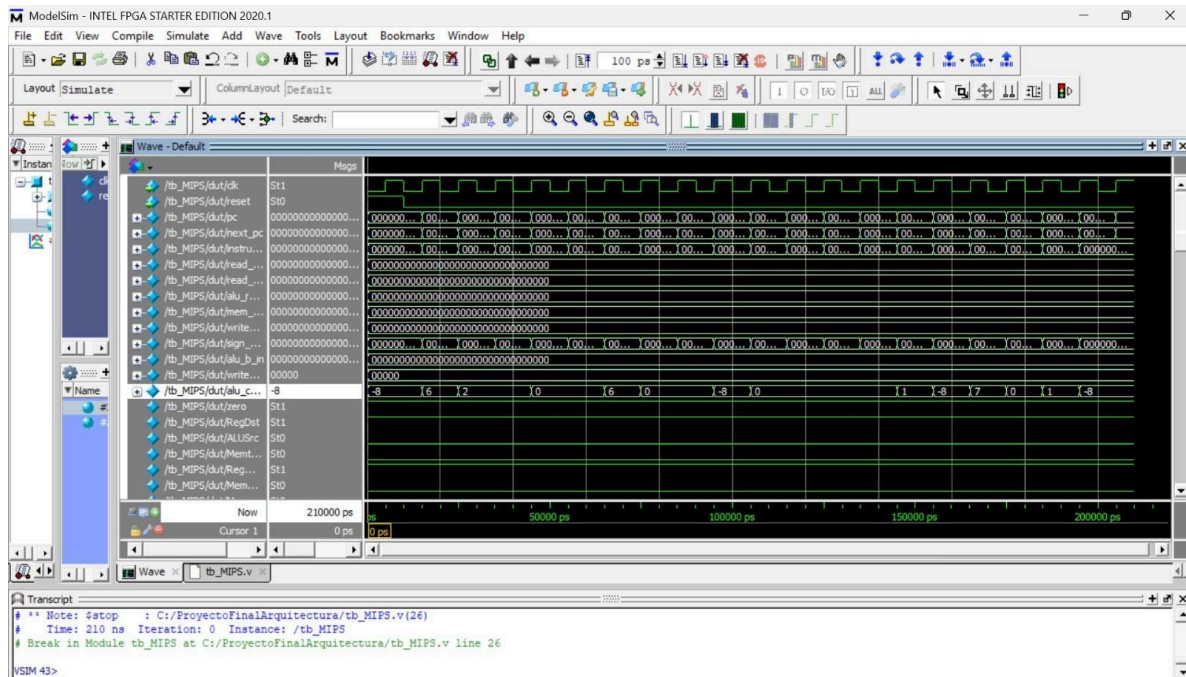


Figura 4. Simulación en ModelSim — vista completa del datapath. Se confirma la correcta escritura en el banco de registros (*write_data*) en cada ciclo activo de *RegWrite*.

4. PROPUESTA DE ALGORITMO EN ENSAMBLADOR

4.1 Justificación de la elección

Para validar el funcionamiento del datapath se propone el Test de Primalidad por Restas Sucesivas, utilizado para determinar si un número es primo. Este algoritmo es útil en teoría de números y aplicaciones de seguridad informática como la generación de llaves criptográficas.

- Utiliza únicamente instrucciones sub (resta), beq (comparación de igualdad), slt (comparación menor que), j (salto incondicional) y operaciones de registro — todas disponibles en el ISA implementado.
- Evita la instrucción de módulo (%) o división, sustituyéndola por restas repetidas.
- Implementa lógica de control mediante un bucle externo (divisores) y un bucle interno (cálculo del residuo).
- Ejercita múltiples rutas del datapath: ALU con sub y slt, comparación con beq y control de flujo (j).
- Es un algoritmo verificable manualmente; por ejemplo, para $n = 5$ el resultado final indica correctamente que el número es primo.

4.2 Descripción del algoritmo

El algoritmo verifica si un número $n > 1$ es primo probando divisores desde 2 hasta $n-1$. Para cada divisor: Se calcula el residuo de $n \div$ divisor mediante restas sucesivas. Si el residuo resulta 0, significa que el número es divisible y por tanto no es primo. Si ningún divisor logra residuo 0, el número se considera primo.

```
# Algoritmo de Euclides - Máximo Común Divisor (MCD) #
# Entradas: $t0 = a (ej. 48), $t1 = b (ej. 18)
# Salida:      $t0 = MCD(a, b)

loop:
    beq  $t0, $t1, end      # Si a == b, terminamos
    slt  $t2, $t1, $t0      # $t2 = 1 si b < a (es decir, a > b)
    beq  $t2, $zero, else   # Si $t2 == 0 (a <= b), ir a else

    sub  $t0, $t0, $t1      # a > b: a = a - b
    j    loop              # Repetir

else:
    sub  $t1, $t1, $t0      # b > a: b = b - a
    j    loop              # Repetir

end:
    # $t0 contiene el MCD
    # add $s0, $t0, $zero   # Guardar resultado en $s0
    # (opcional)
```

4.4 Traza de ejecución — ejemplo MCD(12, 8)

Ciclo	Divisor (\$t4)	Operación	Residuo (\$t2)	$t2 == 0?$	Resultado
1	2	5-2-2	1	No	Continúa
2	3	5 - 3	2	No	Continúa
3	4	5 - 4	1	No	Continúa
4	5	divisor == n	----	----	FIN, Primo

El resultado final para $n = 5$ es $v0 = 1$, indicando correctamente que el número es primo. El algoritmo utiliza instrucciones sub, slt, beq y j, todas compatibles con el datapath implementado.

5. CONCLUSIONES

Diego Israel González Sánchez

El diseño del datapath MIPS de ciclo único en Verilog implicó comprender con precisión cómo cada señal de control influye en el flujo de datos. La parte más desafiante fue asegurar que las señales combinacionales de la unidad de control y el ALUControl produjeran los valores correctos en cada instrucción, ya que un error en el campo funct o en la asignación de alu_control afecta directamente el resultado de la ALU. Esta práctica consolidó la comprensión de cómo el hardware interpreta y ejecuta una instrucción en binario: desde los bits del opcode hasta el resultado escrito en un registro.

Derek Gabriel Casillas López

La experiencia más valiosa fue entender la correspondencia entre el código ensamblador y su representación en código máquina binario de 32 bits. Al codificar manualmente las instrucciones de prueba en el archivo TestF1_MemInst.mem, fue necesario conocer exactamente la posición y el significado de cada campo en la instrucción. Esto ofrece una perspectiva completamente diferente sobre lo que ocurre a nivel de hardware cuando un procesador ejecuta código. La organización en fases progresivas del proyecto facilita construir el conocimiento de manera incremental y sólida.

Iván Samuel Ortiz Orellana

La implementación de los módulos de control fue la parte de mayor cuidado en este proyecto. Asegurarse de que la Unidad de Control generara los valores correctos para RegDst, RegWrite y ALUOp, y que el ALUControl los tradujera correctamente al código de operación de la ALU, requirió revisar la especificación del ISA de forma detallada. El diseño modular de Verilog demostró ser una ventaja, ya que permitió depurar cada bloque de forma independiente antes de integrarlos en MIPS_Top y verificar el sistema completo en ModelSim.

Omar Luna Reyes

Implementar el algoritmo de Euclides en ensamblador MIPS con las restricciones del ISA definido fue un ejercicio que evidencia cómo los compiladores deben adaptar construcciones de alto nivel a las instrucciones disponibles en el hardware. La ausencia de instrucciones de división directa obligó a rediseñar el algoritmo usando restas sucesivas, lo cual es un ejemplo concreto de la relación entre el diseño del ISA y la complejidad de los algoritmos de software. Validar manualmente la traza de ejecución del algoritmo antes de simularlo reforzó la comprensión del flujo de control en MIPS.

Conclusión General

El proyecto demostró que la arquitectura MIPS, a pesar de ser considerada clásica, sigue siendo una plataforma de aprendizaje inigualable por la claridad de su diseño. La implementación del datapath single-cycle en Verilog integró conceptos de lógica digital, organización de computadoras y lenguaje ensamblador en un único proyecto cohesionado. Los módulos implementados sientan la base para las extensiones de Fase 2, donde se agregarán instrucciones tipo I y J, completando un procesador MIPS funcional con mayor repertorio de instrucciones.

6. REFERENCIAS

- [1] D. A. Patterson y J. L. Hennessy, *Computer Organization and Design: The Hardware/Software Interface*, 5.^a ed. Waltham, MA: Morgan Kaufmann, 2014.
- [2] MIPS Technologies, *MIPS32 Architecture for Programmers Volume I: Introduction to the MIPS32 Architecture*. Mountain View, CA: MIPS Technologies, 2001.
- [3] MIPS Technologies, *MIPS32 Architecture for Programmers Volume II: The MIPS32 Instruction Set*. Mountain View, CA: MIPS Technologies, 2001.
- [4] S. Brown y Z. Vranesic, *Fundamentals of Digital Logic with Verilog Design*, 3.^a ed. New York, NY: McGraw-Hill, 2014.
- [5] Intel/Altera, *ModelSim-Altera Software Simulation User Guide*. San Jose, CA: Altera Corporation, 2016.
- [6] D. M. Harris y S. L. Harris, *Digital Design and Computer Architecture*, 2.^a ed. Waltham, MA: Morgan Kaufmann, 2012.